

1. Project Overview

Used Car Market Analytics Dashboard

This project analyzes **1,005,000+ used car records** to identify pricing trends, brand performance, vehicle characteristics, and regional market insights. The objective is to transform raw vehicle data into meaningful business insights using **Python, PostgreSQL, and Power BI**.

2. Dataset Summary

Dataset Information

- **Rows:** 1,005,000+
- **Columns:** 20

Key Features

- Brand
 - Model
 - Year
 - Mileage (km/l)
 - Engine CC
 - Horsepower
 - Fuel Type
 - Transmission
 - Owner Type
 - Color
 - City
 - Kilometers Driven
 - Insurance Status
 - Service History
 - Accident History
 - Registration Age
 - Price
-

3. Data Preparation & Cleaning Using Python

We started the project by preparing and cleaning the dataset in Python.

Data Loading

- Imported the dataset using **Pandas**.
- Loaded CSV into a DataFrame.

Initial Exploration

Performed initial data understanding using

- `df.head()`
- `df.info()`
- `df.describe()`

to understand

- dataset structure
- column data types
- statistical summary

0	Toyota	Camry	2001	19.615852	1396.560379	NaN	Hybrid	
1	Hyundai	i20	2012	19.478608	1130.771005	137.021719	Petrol	
2	Mahindra	Bolero	2008	17.469920	1766.466250	142.902960	NaN	
3	BMW	X1	2015	21.295421	NaN	49.782079	NaN	
4	Honda	Civic	2022	18.326833	1239.334935	121.505993	Diesel	

0	Automatic	First	Grey	Chennai	500928	1	
1	NaN	First	Grey	Mumbai	211510	0	
2	Automatic	First	Black	Kolkata	333999	1	
3	Manual	First	White	Pune	225930	1	
4	Manual	Second	White	Kolkata	43404	1	

0	1	0	1	4	5
1	1	0	1	4	5
2	0	0	1	5	5
3	1	3	1	4	5
4	0	0	1	5	5

0	24	3.531896e+05
1	13	9.596943e+05
2	17	2.920229e+05
3	10	2.949453e+06
4	3	6.365208e+05

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1005000 entries, 0 to 1004999
Data columns (total 20 columns):
Column Non-Null Count Dtype

0 Brand 1005000 non-null object
1 Model 1005000 non-null object
2 Year 1005000 non-null int64
3 Mileage_kmpl 924610 non-null float64
4 Engine_CC 924597 non-null float64
5 Horsepower 924597 non-null float64
6 Fuel_Type 926267 non-null object
7 Transmission 924615 non-null object
8 Owner_Type 1005000 non-null object
9 Color 924581 non-null object
10 City 924570 non-null object
11 Kms_Driven 1005000 non-null int64
12 Insurance_Valid 1005000 non-null int64

Missing Value Handling

Checked missing values using

```
df.isnull().sum()
```

Handled missing values by

Numerical Columns

Filled using **Median**

- Mileage
- Engine CC
- Horsepower
- Registration Age

Categorical Columns

Filled using **Mode**

- Fuel Type
- Transmission
- Color
- City

Binary Columns

Filled using **Mode**

- Insurance
- Service History
- Accidents
- Tax Paid
- Seats
- Number of Doors

```

Transmission      80385
Owner_Type        0
Color             80419
City              80430
Kms_Driven        0
Insurance_Valid   0
Service_History   0
Accidents         0
Tax_Paid          0
Number_of_Doors   0
Seats            0
Registration_Age   0
Price            0
dtype: int64

Duplicate Rows: 5000

Duplicate rows after removal: 0

Null Values After Cleaning:
Brand      0
Model      0
Year       0
Mileage_kmpl 0
Engine_CC  0
Horsepower 0
Fuel_Type  0
Transmission 0
Owner_Type 0
Color      0
City       0
Kms_Driven 0
Insurance_Valid 0
Service_History 0
Accidents 0
Tax_Paid 0
Number_of_Doors 0
Seats 0
Registration_Age 0
Price 0
dtype: int64

```

Duplicate Removal

Checked duplicate records

Removed duplicate rows to improve data quality.

```

Duplicate Rows: 5000

Duplicate rows after removal: 0

Null Values After Cleaning:

```

Data Standardization

Standardized

- Brand Names
- Fuel Types

Example

BMW instead of Bmw

Electric instead of Electrik

Export

Saved the cleaned dataset as

```
cleaned_cars.csv
```

for SQL analysis.

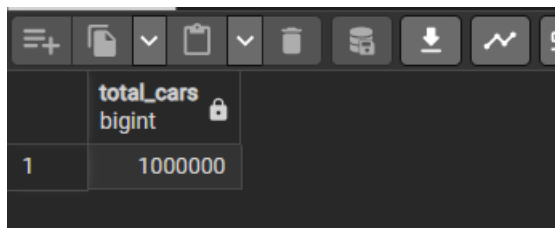
4. Data Analysis Using PostgreSQL

After cleaning the data, the dataset was imported into PostgreSQL for business analysis.

We executed **18 SQL queries** to answer important business questions.

Business Questions

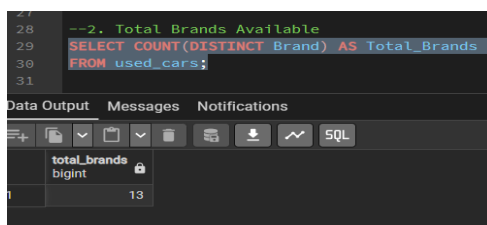
1. Total Cars Available



The screenshot shows the PostgreSQL Data Output window. The query result is a single row with the column name 'total_cars' and data type 'bigint'. The value is 1000000.

	total_cars bigint
1	1000000

2. Total Brands Available

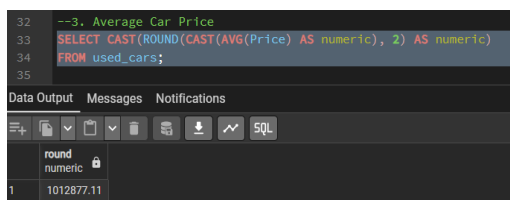


The screenshot shows the PostgreSQL Data Output window. The query result is a single row with the column name 'total_brands' and data type 'bigint'. The value is 13.

```
--2. Total Brands Available
SELECT COUNT(DISTINCT Brand) AS Total_Brands
FROM used_cars;
```

	total_brands bigint
1	13

3. Average Selling Price



The screenshot shows the PostgreSQL Data Output window. The query result is a single row with the column name 'round' and data type 'numeric'. The value is 1012877.11.

```
--3. Average Car Price
SELECT CAST(ROUND(CAST(AVG(Price) AS numeric), 2) AS numeric)
FROM used_cars;
```

	round numeric
1	1012877.11

4. Top 10 Most Expensive Cars

```
--4. Top 10 Most Expensive Cars
SELECT Brand,
       Model,
       CAST(Price AS NUMERIC(12,2)) AS Price
FROM used_cars
ORDER BY Price DESC
LIMIT 10;
```

brand	model	price
Mercedes	GLA	37885997.53
Mercedes	C-Class	37553759.89
Mercedes	GLA	35852892.15
Mercedes	E-Class	34955337.51
Mercedes	C-Class	34866019.12
Mercedes	GLA	34775765.72
Mercedes	E-Class	34574728.32
Mercedes	C-Class	34305029.61
BMW	X1	33831517.15
BMW	3 Series	33485533.96

5. Brand-wise Average Price

```
45 SELECT Brand,
46        ROUND(AVG(Price)::numeric, 2) AS Avg_Price
47 FROM used_cars
48 GROUP BY Brand
49 ORDER BY Avg_Price DESC;
```

brand	avg_price
Mercedes	2514847.75
BMW	2191407.95
Audi	2082239.77
Volkswagen	792344.53
Skoda	745284.42
Kia	744586.85
Honda	650913.55
Toyota	647762.25
Hyundai	605570.04
Mahindra	558590.24
Nissan	558406.77
Ford	555259.62
Tata	513798.20

6. Most Popular Brands

```
51 --6. Most Popular Car Brands
52 SELECT Brand,
53        COUNT(*) AS Total_Cars
54 FROM used_cars
55 GROUP BY Brand
56 ORDER BY Total_Cars DESC;
```

brand	total_cars
BMW	77169
Skoda	77157
Hyundai	77141
Ford	77057
Toyota	76996
Mercedes	76938
Audi	76937
Honda	76927
Kia	76927
Volkswagen	76783
Mahindra	76708
Tata	76672
Nissan	76588

7. Fuel Type Distribution

```

57
58 --7. Fuel Type Distribution
59 SELECT Fuel_Type,
60        COUNT(*) AS Total_Cars
61 FROM used_cars
62 GROUP BY Fuel_Type;
63

```

	fuelType character varying (20)	total_cars bigint
1	CNG	90190
2	Diesel	277382
3	Electric	75394
4	Hybrid	66461
5	Petrol	490573

8. Transmission Distribution

```

64 --8. Transmission Type Distribution
65 SELECT Transmission,
66        COUNT(*) AS Total_Cars
67 FROM used_cars
68 GROUP BY Transmission;
69

```

	transmission character varying (20)	total_cars bigint
1	Automatic	322352
2	Manual	677648

9. City-wise Car Listings

```

70 --9. City-wise Car Listings
71 SELECT City,
72        COUNT(*) AS Total_Cars
73 FROM used_cars
74 GROUP BY City
75 ORDER BY Total_Cars DESC;
76

```

	city character varying (50)	total_cars bigint
	Delhi	195417
	Hyderabad	115250
	Mumbai	115203
	Pune	115018
	Ahmedabad	114930
	Bangalore	114883
	Kolkata	114867
	Chennai	114432

10. City-wise Average Price

Query	Query History
77	--10. City-wise Average Price
78	SELECT City,
79	ROUND(AVG(Price)::numeric, 2) AS Avg_Price
80	FROM used_cars
81	GROUP BY City
82	ORDER BY Avg_Price DESC;
83	
Data Output	Messages
	SQL
city	avg_price
character varying (50)	numeric
1 Mumbai	1016669.63
2 Delhi	1016093.53
3 Ahmedabad	1014591.99
4 Chennai	1012633.38
5 Pune	1011401.86
6 Kolkata	1010494.81
7 Hyderabad	1009633.43
8 Bangalore	1009243.06

11. Average Price by Fuel Type

84	--11. Average Price by Fuel Type
85	SELECT Fuel_Type,
86	ROUND(AVG(Price)::numeric, 2) AS Avg_Price
87	FROM used_cars
88	GROUP BY Fuel_Type
89	ORDER BY Avg_Price DESC;
90	
Data Output	Messages
	SQL
fuel_type	avg_price
character varying (20)	numeric
CNG	1015641.55
Hybrid	1014984.44
Electric	1013415.27
Petrol	1012465.11
Diesel	1012055.72

12. Average Price by Transmission

```
--12. Average Price by Transmission
SELECT Transmission,
       ROUND(AVG(Price)::numeric, 2) AS Avg_Price
FROM used_cars
GROUP BY Transmission;
```

Output

Messages

Notifications

</

13. Insurance Status Analysis

```

999 --1. Cars with Valid Insurance
1000 SELECT Insurance_Valid,
1001        COUNT(*) AS Total_Cars
1002 FROM used_cars
1003 GROUP BY Insurance_Valid;
1004

```

Data Output		Messages	Notifications
			SQL
insurance_valid	total_cars		
character varying (20)	bigint		
0	150532		
1	849468		

14. Accident Impact on Price


```

104
105 --14. Accident Impact on Price
106 SELECT Accidents,
107        ROUND(AVG(Price)::numeric, 2) AS Avg_Price
108 FROM used_cars
109 GROUP BY Accidents;
110

```

	accidents character varying (20)	avg_price numeric
1	0	1018914.06
2	1	999415.82
3	2	979200.08
4	3	911251.90
5	4	926034.86
6	5	645636.11

15. Top 5 Cities with Highest Average Price

```

--15. Top 5 Cities with Highest Average Price
SELECT City,
       ROUND(AVG(Price)::numeric, 2) AS Avg_Price
FROM used_cars
GROUP BY City
ORDER BY Avg_Price DESC
LIMIT 5;

```

city character varying (50)	avg_price numeric
Mumbai	1016669.63
Delhi	1016093.53
Ahmedabad	1014591.99
Chennai	1012633.38
Pune	1011401.86

16. Registration Age Analysis

	car_age integer	avg_price numeric
2	3	1531400.01
3	4	1495772.69
4	5	1429976.77
5	6	1380858.55
6	7	1337193.51
7	8	1290231.12
8	9	1240814.55
9	10	1188862.70
10	11	1136555.21
11	12	1082141.95
12	13	1040963.62
13	14	987109.41
14	15	941025.47
15	16	892497.24
16	17	849136.98
17	18	802320.84
18	19	774241.22
19	20	722234.98
20	21	687628.72
21	22	641180.46
22	23	617787.09
23	24	581228.04
24	25	553559.50
25	26	529593.72

17. Brand Popularity by City

```

127
128 --17. Top Brand in Each City
129 SELECT City, Brand, COUNT(*) AS Total_Cars
130 FROM used_cars
131 GROUP BY City, Brand
132 ORDER BY City, Total_Cars DESC;

```

	city character varying (50)	brand character varying (50)	total_cars bigint
1	Ahmedabad	Toyota	9004
2	Ahmedabad	Audi	8908
3	Ahmedabad	BMW	8906
4	Ahmedabad	Mercedes	8883
5	Ahmedabad	Kia	8867
6	Ahmedabad	Nissan	8860
7	Ahmedabad	Mahindra	8859
8	Ahmedabad	Honda	8825
9	Ahmedabad	Tata	8799
10	Ahmedabad	Skoda	8791
11	Ahmedabad	Hyundai	8779
12	Ahmedabad	Volkswagen	8734
13	Ahmedabad	Ford	8715
14	Bangalore	Tata	8936
15	Bangalore	Hyundai	8926
16	Bangalore	Audi	8896
17	Bangalore	Ford	8869

18. Average Mileage by Fuel Type

```

--18. Average Mileage by Fuel Type
SELECT Fuel_Type,
       ROUND(AVG(Mileage_kmpl)::numeric, 2) AS Avg_Mileage
FROM used_cars
GROUP BY Fuel_Type;

```

fuel_type character varying (20)	avg_mileage numeric
CNG	17.99
Diesel	17.99
Electric	17.99
Hybrid	18.01
Petrol	18.00

5. Dashboard in Power BI

Finally, the cleaned and analyzed data was connected to Power BI to build an interactive dashboard for business users.

Three dashboards were developed.

Dashboard 1 — Executive Dashboard

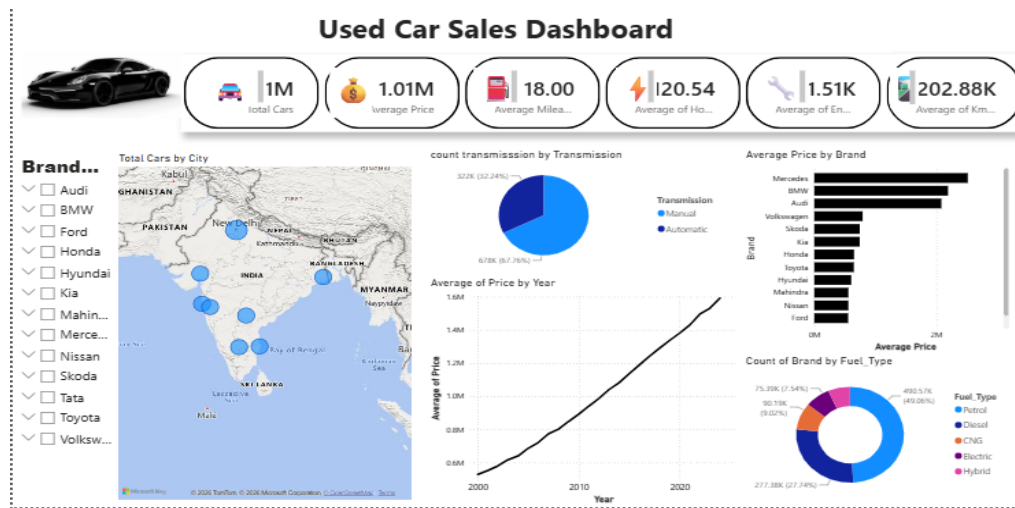
This dashboard provides a complete overview of the used car market.

KPIs

- Total Cars
- Average Price
- Average Mileage
- Average Horsepower
- Average Engine Capacity
- Average Kilometers Driven

Visualizations

- City-wise Map
- Fuel Type Distribution
- Transmission Distribution
- Brand-wise Average Price
- Price Trend by Manufacturing Year

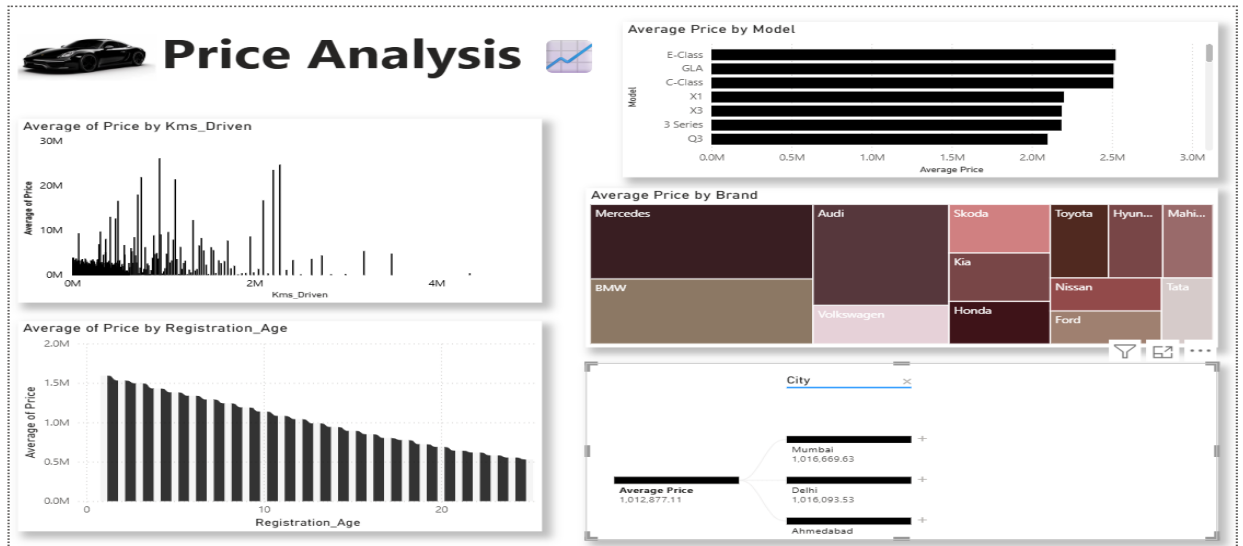


Dashboard 2 — Price Analysis Dashboard

This dashboard focuses on vehicle pricing.

Visualizations

- Average Price by Model
- Average Price by Brand
- Average Price by Fuel Type
- Average Price by Registration Age
- Average Price by Kilometers Driven
- City-wise Price Analysis

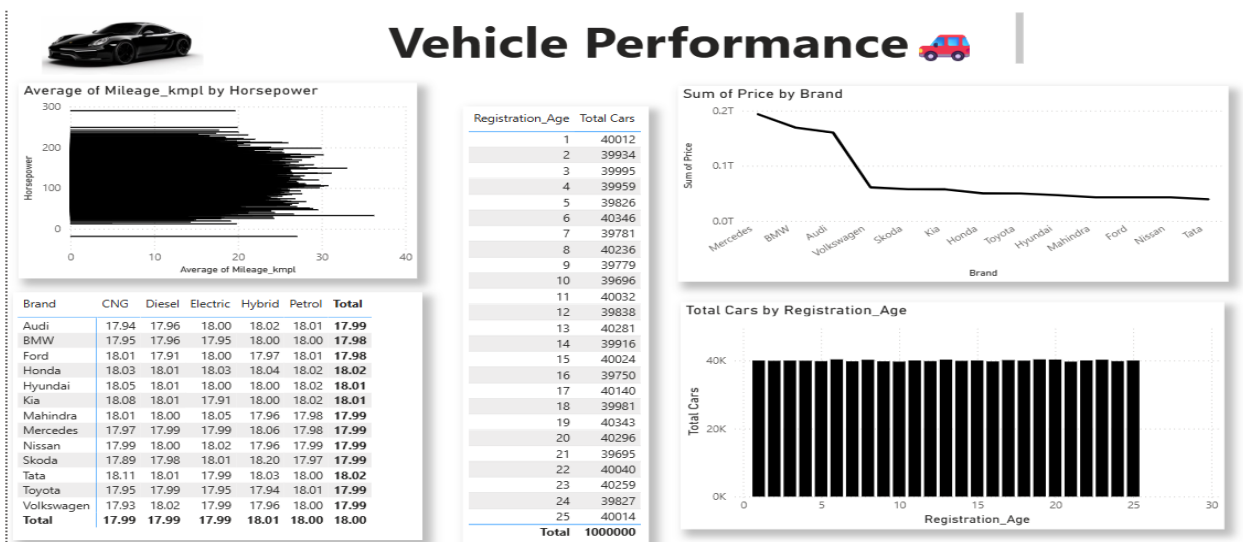


Dashboard 3 — Vehicle Performance Dashboard

This dashboard evaluates vehicle characteristics.

Visualizations

- Mileage vs Horsepower
- Registration Age Distribution
- Total Cars by Registration Age
- Brand-wise Total Market Value
- Mileage Comparison by Fuel Type



6. Key Business Insights

- Mercedes has the highest average resale value.
- BMW has the highest number of listed vehicles.
- Petrol vehicles dominate the used car market.
- Manual transmission cars are more common than automatic cars.
- Mumbai has the highest average selling price.
- Vehicle price decreases as registration age increases.
- Average mileage remains close to **18 km/l** across brands.

7. Business Recommendations

- Focus premium marketing campaigns on luxury brands such as Mercedes, BMW, and Audi.
- Increase inventory in cities with higher demand and selling prices.
- Promote fuel-efficient vehicles to attract cost-conscious customers.
- Encourage proper vehicle maintenance to improve resale value.
- Use regional pricing insights to optimize pricing strategies across cities.

8. Conclusion

This project successfully demonstrates an **end-to-end Data Analytics workflow** using **Python, PostgreSQL, and Power BI**.

Project Achievements

- Cleaned and processed **1 million+ records**
- Performed business analysis using **18 SQL queries**
- Built **3 interactive Power BI dashboards**
- Generated actionable business insights to support data-driven decision making